

W4111 — Spring 2026: Exam 3 Sample Questions & Answers

Combined study packet — questions from `exam-3-sample-questions.md`, answers from `exam-3-sample-questions-opus-answers.md`. Answers are based on Lectures 9–12 (NoSQL wrap-up, normalization, query processing, transactions / concurrency / recovery, scaling, REST, big data / data engineering / warehousing).

Note on this document. These are example short-answer questions. They are *not* representative of the exam as a whole or of the exam's length — they are simply examples of what the short written questions may look like. Each question begins on a new page so the table of contents below has predictable page numbers when printed.

Table of Contents

#	Topic	Page
Q1	Functional Dependencies	3
Q2	3NF vs. BCNF	5
Q3	Join Algorithms (descriptions)	6
Q4	Index Costs / Join Algorithms	7
Q5	Materialization vs. Pipelining	8
Q6	Logical vs. Physical Plans	9
Q7	Conjunctive vs. Disjunctive Selection	10
Q8	Join Algorithm Selection	11
Q9	Transaction Execution	12
Q10	Isolation Levels	13
Q11	ACID vs. BASE	14
Q12	Concurrency Control (Strict 2PL)	15
Q13	Scaling and Architecture	16
Q14	Serializability	17
Q15	Cascading Rollbacks	18
Q16	Strict Two-Phase Locking	19
Q17	Resource-Oriented Design (REST)	20
Q18	Data Engineering Pipeline (ETL / ELT)	21
Q19	Deadlock Prevention vs. Detection	22
Q20	Star Schema Design	23

*Total: 23 pages (Letter, 0.85" margins). Q1 spans 2 sheets; all other questions fit on one sheet each.
Footer at the bottom of every page shows the current page number.*

Q1 — Functional Dependencies

Consider the relation:

$R(A, B, C, D, E)$

The following sample tuples are given:

A	B	C	D	E
a1	b1	c1	d1	e1
a1	b1	c2	d1	e2
a2	b1	c1	d2	e3
a2	b1	c2	d2	e4
a3	b2	c1	d3	e5
a3	b2	c2	d3	e6

Questions

1. Identify all **non-trivial functional dependencies** that hold on this relation based on the data shown.
2. For each of the following, state whether the FD holds, with brief justification: (a) $A \rightarrow D$ (b) $B \rightarrow A$ (c) $A \rightarrow B$ (d) $(A, C) \rightarrow E$
3. Find a **candidate key** for this relation. Show your reasoning.
4. Does the functional dependency $C \rightarrow E$ hold? Why or why not?

Instructions: base your answers only on the data; justify each answer using the tuples; partial credit is given for correct reasoning.

Answer

1. Non-trivial FDs that hold on the data:

- $A \rightarrow B$: $a1 \rightarrow b1$, $a2 \rightarrow b1$, $a3 \rightarrow b2$ (each A maps to exactly one B).
- $A \rightarrow D$: $a1 \rightarrow d1$, $a2 \rightarrow d2$, $a3 \rightarrow d3$.
- $D \rightarrow A$: $d1 \rightarrow a1$, $d2 \rightarrow a2$, $d3 \rightarrow a3$ (so $A \leftrightarrow D$).
- $D \rightarrow B$ (transitively from $D \rightarrow A \rightarrow B$).
- $(A, C) \rightarrow E$: every (A, C) pair is unique and maps to a unique E.
- Combined: $A \rightarrow BD$ and $(A, C) \rightarrow BDE$ (and equivalently $(D, C) \rightarrow ABE$).

2.

- (a) $A \rightarrow D$ — **holds**. Each A value pairs with exactly one D.
- (b) $B \rightarrow A$ — **does NOT hold**. $B = b1$ pairs with both $a1$ and $a2$, violating uniqueness.
- (c) $A \rightarrow B$ — **holds**. Each A value maps to exactly one B.

- (d) $(A, C) \rightarrow E$ — **holds**. Every (A, C) combination determines a unique E .

3. Candidate key: (A, C) — equivalently (D, C) .

A alone determines only B and D , not C or E . C alone has only two distinct values and can't be a key. Together, $(A, C) \rightarrow BDE$, so $(A, C)^+ = ABCDE$. Neither A nor C is a superkey alone, so (A, C) is minimal — hence a candidate key. Because $A \leftrightarrow D$, (D, C) is also a candidate key.

4. $C \rightarrow E$ — does NOT hold. $C = c1$ maps to multiple distinct E values ($e1, e3, e5$), so C does not functionally determine E .

Q2 — 3NF vs. BCNF

Consider the relation $R(A, B, C)$ with FDs:

- $A \rightarrow B$
- $B \rightarrow C$

Questions

1. What are the **candidate key(s)** for R ? Show your reasoning.
2. Is the relation R in **Third Normal Form (3NF)**? Explain why or why not.
3. Is the relation R in **Boyce-Codd Normal Form (BCNF)**? Explain why or why not.
4. If R is not in BCNF, decompose R into a set of relations that are in BCNF. Show the resulting schemas.

Instructions: clearly justify each answer; you may use the formal definitions of 3NF and BCNF.

Answer

1. Candidate key: $\{A\}$. $A^+ = \{A, B, C\}$. $B^+ = \{B, C\}$ (no A). $C^+ = \{C\}$. So only A is a candidate key. The single prime attribute is A .

2. Is R in 3NF? No.

3NF requires that for every non-trivial FD $X \rightarrow Y$, either X is a superkey OR every attribute in $Y - X$ is a prime attribute (part of some candidate key).

- $A \rightarrow B$: A is a superkey. ✓
- $B \rightarrow C$: B is *not* a superkey, and C is *not* a prime attribute. ✗

This is the classic transitive dependency ($A \rightarrow B \rightarrow C$), so R is in 2NF but not 3NF.

3. Is R in BCNF? No.

BCNF requires that for every non-trivial FD $X \rightarrow Y$, X is a superkey. $B \rightarrow C$ violates this since B is not a superkey.

4. BCNF decomposition:

- $R_1(A, B)$ with FD $A \rightarrow B$
- $R_2(B, C)$ with FD $B \rightarrow C$

Both are in BCNF (the LHS of every FD is the key of its relation). The decomposition is lossless (the common attribute B is a key of R_2) and dependency-preserving.

Q3 — Join Algorithms

Consider joining two relations **R** and **S**.

1. Briefly describe: - Nested-loop join - Indexed nested-loop join - Merge join - Hash join
 2. Under what conditions is each algorithm preferred?
 3. Why does the choice of **outer vs. inner relation** matter in nested-loop joins?
-

Answer

1. Descriptions:

- **Nested-loop join (NLJ):** For each tuple of outer R, scan all of S to find matches. Cost $\approx |R| \cdot |S|$ (block-NLJ improves this using buffer pages).
- **Indexed nested-loop join:** Like NLJ, but for each outer tuple use an index on `S.join_attr` to find matches directly. Cost $\approx |R| \cdot (\text{cost of one index lookup})$.
- **Merge (sort-merge) join:** Sort both relations on the join key, then make a single linear pass over both, advancing pointers. Cost $\approx \text{sort}(R) + \text{sort}(S) + |R| + |S|$.
- **Hash join:** Build an in-memory hash table on the smaller relation's join key, then probe it with each tuple of the other relation. Cost $\approx |R| + |S|$ (when build side fits in memory).

2. When each is preferred:

- **NLJ:** small inputs, no useful indexes, or non-equijoins (e.g., `R.x < S.y`).
- **Indexed NLJ:** when one relation is much smaller and the larger one has an index on the join attribute.
- **Merge join:** when both inputs are already sorted on the join key (e.g., from a previous operator or a clustered index), or when the join needs to produce ordered output.
- **Hash join:** equijoins on large unsorted/unindexed inputs where the build side fits (or can be partitioned to fit) in memory.

3. Why outer/inner choice matters: The outer is read once; the inner is read (or probed) once per outer tuple. Choosing the smaller relation as the outer and exploiting an index/buffering on the inner minimizes total I/O. With block-nested loops, having the smaller relation as the outer also reduces the number of times the inner is rescanned.

Q4 — Index Costs

Assume a relation R has N records.

1. Expressed as order notation in N , what is the approximate complexity / cost of: (a) Full table scan (b) B+-tree index lookup (c) Hash lookup
2. Why might building an index be beneficial even if it has a cost?
3. Explain why a query plan that is optimal for one operation may not be optimal for the entire query.

Answer

1. Order-notation costs to find a record in R with N tuples:

- (a) **Full table scan:** $O(N)$
- (b) **B+-tree index lookup:** $O(\log N)$ (height of the tree)
- (c) **Hash lookup:** $O(1)$ average (equality only)

2. Why build indexes despite their cost?

Indexes are built once but used many times. A B+-tree turns repeated $O(N)$ scans into $O(\log N)$ lookups, dramatically cutting total work for selective queries, joins (indexed NLJ), and **ORDER BY**. The index also enables physical operators (e.g., index nested-loop join, index range scans) that would be impossible otherwise. The build cost amortizes across many subsequent queries.

3. Why local optima $\neq$ global optima:

A query is a tree of operators. Choosing the cheapest algorithm for one operator can produce intermediate results that are large, unsorted, or unhashed — increasing the cost of later operators (e.g., sort-merge needs sorted input; hash join wants the smaller side as build). The optimizer must consider sort orders, pipelined vs. materialized output, and cumulative I/O — so the globally optimal plan may use non-locally-optimal operators that cooperate well with their neighbors.

Q5 — Materialization vs. Pipelining

1. Define: (a) Materialization (b) Pipelining
2. What are the advantages and disadvantages of each?
3. Why can pipelining not always be used?

Answer

1. Definitions:

- (a) **Materialization**: Each operator computes its full result and writes it to a temporary relation (in memory or on disk) before the next operator reads it.
- (b) **Pipelining**: An operator passes each output tuple immediately to the next (parent) operator without writing the entire intermediate result first ("tuple-at-a-time" or iterator/Volcano model).

2. Pros and cons:

- **Materialization**: + Simple; supports any operator, including blocking ones; intermediate results can be reused. – Extra I/O to write/read temporaries; higher memory; latency until first result is available.
- **Pipelining**: + Avoids writing intermediates → less I/O, lower latency to the first row, less temp storage. – Requires non-blocking operators; harder to reuse results; can hold many operators open in memory simultaneously.

3. Why pipelining isn't always usable: Some operators are **blocking** — they must consume all of their input before producing any output: sorting, hash aggregation / `GROUP BY`, the build phase of hash join, set difference, and many `DISTINCT` / `ORDER BY` operations. These force materialization at that point in the plan.

Q6 — Logical vs. Physical Plans

Consider the query:

```
SELECT name FROM instructor WHERE salary < 75000;
```

1. Write two equivalent relational-algebra expressions.
 2. Which one is likely more efficient? Why?
-

Answer

1. Two equivalent relational-algebra expressions:

- (a) $\pi_{\text{name}}(\sigma_{\text{salary} < 75000}(\text{instructor}))$ — *select first, then project*
- (b) $\sigma_{\text{salary} < 75000}(\pi_{\text{name, salary}}(\text{instructor}))$ — *project first (must keep `salary` so the predicate can be evaluated), then select*

(Or, combining both: $\pi_{\text{name}}(\sigma_{\text{salary} < 75000}(\pi_{\text{name, salary}}(\text{instructor})))$.)

2. Which is more efficient? Form (a) — **selection pushed down before projection** — is generally more efficient. The selection reduces the number of tuples first, so the projection then runs over fewer rows. If an index on `salary` exists, (a) can also use it directly, whereas projecting first prevents that. This is the classic heuristic "push selections down."

Q7 — Conjunctive vs. Disjunctive Selection

Assume:

- An index exists on `name`
- No index on `dept_name`

Compare the performance of:

1. `WHERE name = 'Smith' AND dept_name = 'CS'`
2. `WHERE name = 'Smith' OR dept_name = 'CS'`

Why does the index help in one case but not the other?

Answer

1. **AND:** `WHERE name = 'Smith' AND dept_name = 'CS'`

The optimizer can use the index on `name` to fetch only rows where `name = 'Smith'`, then test `dept_name = 'CS'` on that small set. Very fast.

2. **OR:** `WHERE name = 'Smith' OR dept_name = 'CS'`

The result must include every row that satisfies *either* predicate. Even if we use the index on `name`, we still need to find rows with `dept_name = 'CS'` among the rows that didn't match — which requires a **full table scan** since no index exists on `dept_name`. The index gives no benefit.

Why: AND is *restrictive* — narrowing on any one indexed predicate also narrows the final answer set. OR is *expansive* — every disjunct must be evaluated, so missing an index on any disjunct forces a full scan.

Q8 — Join Algorithm Selection

Given:

- R has 1,000 tuples
- S has 1,000,000 tuples
- An index exists on `S.join_attr`

1. Which join algorithm is most appropriate?
2. Which relation should be the outer relation? Why?

Answer

1. Best algorithm: Indexed nested-loop join with R as the outer. For each of the 1,000 R tuples, probe the index on S $\rightarrow$ only $\sim 1,000$ index lookups ($\approx 1,000 \cdot \log |S|$), avoiding scanning the 1M-row S. Hash join would also work but requires more memory and offers little benefit when $|R|$ is tiny and an index is already available.

2. Outer relation: R (the small one). This minimizes the number of probes into the indexed relation S. Reversing it (S outer, R inner) would require 1,000,000 lookups — a thousand-fold worse — and would not exploit the index on S.

Q9 — Transaction Execution

Consider a transaction that transfers \$50 from account A to account B.

1. What can go wrong if the transaction fails after updating A but before updating B?
 2. Which ACID property is violated?
 3. How does the DBMS prevent this problem?
-

Answer

Transfer \$50: $A := A - 50$; $B := B + 50$.

1. What goes wrong if it fails after updating A but before B?

A has been debited \$50, but B was never credited. Money disappears from the system; the database is in an inconsistent state.

2. ACID property violated: Atomicity (and consequently consistency). Atomicity says a transaction is "all or nothing" — partial effects must not be visible.

3. How the DBMS prevents it: Through the **write-ahead log (WAL)** and recovery. Before applying a change, the DBMS logs the old/new values. On crash, recovery uses the log to **roll back** any incomplete transaction (undo) and **redo** committed ones, restoring atomicity. A transaction is only considered committed after its commit log record is durably written.

Q10 — Isolation Levels

Compare the following isolation levels:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

For each, state one anomaly that can occur.

Answer

Level	Allows	Example anomaly
Read Uncommitted	Reads of uncommitted data	Dirty read (reads a value another txn later rolls back)
Read Committed	Only committed data; locks released right after read	Non-repeatable read (re-reading the same row gives a different value)
Repeatable Read	Reads stable for the txn's duration	Phantom read (a range query returns new rows on re-execution)
Serializable	Equivalent to some serial schedule	None of the above (full isolation; lowest concurrency)

Q11 — ACID vs. BASE

1. What does BASE stand for?
2. How does BASE differ from ACID?
3. When might a system prefer BASE over ACID?

Answer

1. BASE = Basically Available, Soft state, Eventually consistent.

2. Differences from ACID:

- ACID guarantees strong consistency, isolation, and durability per transaction; replicas (if any) appear identical at all times.
- BASE relaxes consistency: the system stays available under partitions/failures (CAP), accepts that state may be temporarily inconsistent ("soft"), and converges to a consistent state given time ("eventual consistency"). No transactional isolation guarantees.

3. When to prefer BASE: Large-scale distributed systems (web-scale NoSQL, content delivery, social feeds, shopping-cart-style data) where:

- High availability and partition tolerance matter more than instant consistency,
- Read/write throughput must scale horizontally across many nodes/regions,
- Brief inconsistency is acceptable (likes, view counts, recommendations, sessions).

Q12 — Concurrency Control (Strict 2PL)

1. What is **Strict Two-Phase Locking (Strict 2PL)**?
2. Why does it guarantee serializability?
3. What problem does it help prevent during transaction abort?

Answer

1. Strict 2PL: A locking protocol where:

- A transaction acquires the appropriate lock (S/X) before reading/writing,
- Once it releases a lock, it can acquire no new ones (the 2PL "growing/shrinking" rule),
- **Strict part:** all **exclusive (write) locks are held until commit or abort.**

2. Why it guarantees serializability: Two-phase locking produces a **conflict-serializable** schedule because conflicting operations on the same item are forced to be ordered by lock acquisition; the equivalent serial order is given by the order of lock-points (the moment each transaction first releases a lock).

3. Problem prevented during abort: cascading aborts (cascading rollbacks). Because writes aren't visible to others until the writer commits, no other transaction can have read its uncommitted data, so aborting one transaction never forces aborting others — making recovery dramatically simpler.

Q13 — Scaling and Architecture

1. What is the difference between: - scale-up - scale-out
2. What is a **shared-nothing architecture**?
3. Why is scale-out challenging for relational operations like joins?

Answer

1. Scale-up vs. scale-out:

- **Scale-up (vertical):** Make one machine bigger (more CPU/RAM/disk). Easy, but cost grows non-linearly and there is a hardware ceiling.
- **Scale-out (horizontal):** Add more machines and partition/replicate data across them. Near-linear capacity growth, but introduces distributed-systems complexity.

2. Shared-nothing architecture: Each node has its own private CPU, memory, and disk; nodes communicate only over the network. There is no shared memory or shared disk. This is the standard for scale-out (e.g., Hadoop/Spark, MongoDB shards, Cassandra, BigQuery). It eliminates contention on shared resources, enabling horizontal scalability.

3. Why scale-out is hard for joins (and other relational ops): Joins require co-locating matching rows. In a partitioned system, matching keys may live on different nodes, so the engine must **reshuffle/repartition** data across the network (broadcast or hash-redistribute), which is expensive. Cross-node coordination, unequal data distribution (skew), and maintaining ACID over a distributed commit (2PC) further complicate things.

Q14 — Serializability

1. Define **serializability**.
2. Why is it considered the main correctness criterion?
3. Explain the difference between: - conflict serializability - view serializability

Answer

1. Definition: A concurrent schedule is **serializable** if its effect is equivalent to some serial (one-after-another) execution of the same transactions.

2. Why it's the main correctness criterion: Each transaction is written assuming serial semantics — it preserves consistency when run alone. If the concurrent schedule is equivalent to *some* serial order, every transaction still preserves consistency. Anything weaker risks anomalies (lost updates, dirty reads, etc.).

3. Conflict vs. view serializability:

- **Conflict serializability:** A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations. Equivalently, its **precedence (conflict) graph is acyclic**. Easy to check; what 2PL enforces.
- **View serializability:** A weaker, semantic notion: the schedule and some serial schedule produce the same final state and the same reads-from relationships, but the two may not be reachable via swaps. View serializability is a strict superset of conflict serializability and is NP-hard to test in general, so DBMSs use the stricter conflict-serializability in practice.

Q15 — Cascading Rollbacks

1. What is a cascading rollback?
2. Why is it undesirable?
3. How do **cascadeless schedules** prevent this?

Answer

1. **Cascading rollback:** When transaction T1 aborts after another transaction T2 has already read a value T1 wrote, T2 must also be aborted; if T3 read from T2, T3 must abort, and so on.
2. **Why undesirable:** A single failure can roll back a chain of otherwise-successful transactions, wasting work, hurting throughput, and complicating recovery and user expectations.
3. **How cascadeless schedules prevent it:** A **cascadeless schedule** allows a transaction to read a value only **after** the transaction that wrote it has committed. So no transaction ever reads uncommitted data; if any writer aborts, nobody else has been infected. Strict 2PL produces cascadeless schedules.

Q16 — Strict Two-Phase Locking

1. Describe the rules of **Strict 2PL**.
2. Why does it guarantee serializability?
3. How does it simplify recovery?

Answer

1. Rules of Strict 2PL:

- Acquire shared (S) lock before reading; exclusive (X) lock before writing.
- Once any lock is released, no new locks may be acquired (two-phase).
- **Hold all exclusive (write) locks until the transaction commits or aborts** (the "strict" condition; "rigorous 2PL" extends this to *all* locks).

2. Why it guarantees serializability: Two-phase locking is sufficient for conflict serializability because conflicting accesses on the same data item are serialized by the lock manager — you can pick the lock-points to define an equivalent serial order, so the conflict graph is acyclic.

3. How it simplifies recovery: Because writes are not exposed until commit, no other transaction can read or overwrite uncommitted values. This guarantees a **recoverable, cascadeless schedule** — aborting one transaction does not require aborting any others, dramatically simplifying log-based recovery.

Q17 — Resource-Oriented Design

Given entities:

- Course
- Section
- Participant

1. Design REST endpoints for: - retrieving all sections of a course - retrieving all participants in a section
2. Explain the difference between: - a resource - a collection resource

Answer

1. REST endpoints:

- All sections of a course: `GET /courses/{courseId}/sections` (single section: `GET /courses/{courseId}/sections/{sectionId}`)
- All participants in a section: `GET /courses/{courseId}/sections/{sectionId}/participants` (single participant: `.../participants/{participantId}`)

These are hierarchical / nested resource URIs reflecting the containment relationships.

2. Resource vs. collection resource:

- A **resource** is an individual addressable entity, identified by a URI, e.g. `/courses/4111` — a *single* course. Operations target one item (`GET` reads it, `PUT` replaces it, `DELETE` removes it).
- A **collection resource** is itself a resource whose representation is a *set* of resources, e.g. `/courses` or `/courses/4111/sections` . `GET` returns the list, and `POST` typically creates a new member of the collection.

Q18 — Data Engineering Pipeline

1. What are the steps in ETL?
2. What is the difference between ETL and ELT?
3. Why is data engineering often the most time-consuming part of analytics?

Answer

1. ETL steps:

- **Extract** — pull raw data from source systems (OLTP databases, logs, APIs, files).
- **Transform** — clean, deduplicate, conform types, enrich, aggregate, and shape data into the target schema (e.g., a star schema).
- **Load** — write the transformed data into the target store (data warehouse / mart).

2. ETL vs. ELT:

- **ETL** transforms data *before* loading, typically on a separate processing tier; the warehouse stores already-cleaned data.
- **ELT** loads raw data into a powerful target system (cloud DW or data lake — Snowflake, BigQuery, Spark) *first*, then performs transformations inside that system using its own compute. ELT leverages cheap storage + scalable cloud compute and supports schema-on-read; it is the modern default.

3. Why data engineering dominates analytics time: Real-world data is dirty, inconsistent, distributed across many heterogeneous systems, and constantly changing. Tasks like extracting, schema-mapping, deduping, handling missing/late data, and reconciling business definitions are far more time-consuming than the modeling/visualization phase. The often-cited "80% of analytics time is spent on data prep" is a direct consequence.

Q19 — Deadlock Prevention vs. Detection

1. What is the difference between: - deadlock prevention - deadlock detection
2. Explain one prevention technique: - wait-die - wound-wait - timeout
3. What is a **wait-for graph**, and what does a cycle indicate?

Answer

1. Difference:

- **Prevention** stops deadlocks from ever forming, by restricting how/when transactions may wait or acquire locks.
- **Detection** lets deadlocks occur, periodically inspects the system to find them, and resolves by aborting (and later restarting) one or more victims.

2. Prevention technique — Wait-Die (timestamp-based):

Assign each transaction a timestamp at start. When T_i requests a lock held by T_j :

- If T_i is **older** (smaller timestamp), T_i is allowed to **wait**.
- If T_i is **younger**, T_i is **aborted ("dies")** and restarted later with the *same* timestamp.

This guarantees no cycles, since waits go only from older $\rightarrow$ younger.

(*Wound-Wait* is the dual: older transactions "wound" — abort — younger ones; younger ones wait. *Timeout* simply aborts any transaction that waits longer than a threshold.)

3. Wait-for graph: A directed graph with one node per active transaction and an edge $T_i \rightarrow T_j$ whenever T_i is waiting for a lock held by T_j . **A cycle indicates a deadlock** — every transaction in the cycle is waiting for another in the cycle, so none can proceed. The DBMS breaks the cycle by aborting a chosen victim.

Q20 — Star Schema Design

You are designing a data warehouse for sales.

1. What is a **fact table**?
 2. What are **dimension tables**?
 3. Give an example of: - one fact - two dimensions
 4. Why is this model useful for analytics?
-

Answer

1. Fact table: A central table that records measurable business **events** / **metrics** ("facts") at a chosen grain. Rows are typically thin and numeric (measures + foreign keys to dimensions); they grow to billions of rows.

2. Dimension tables: Surrounding tables that provide **descriptive context** for the facts (the "by" attributes — by customer, by product, by date, by store). They are wider (many text/categorical attributes), smaller, and denormalized for query speed.

3. Example for a sales warehouse:

- **Fact (one):** `fact_sales(sale_id, date_key, product_key, store_key, customer_key, units_sold, revenue, discount)`
- Example facts/measures: `units_sold`, `revenue`.
- **Dimensions (two):**
- `dim_product(product_key, name, category, brand, package_size, ...)`
- `dim_date(date_key, day, month, quarter, year, weekday, holiday_flag, ...)`

4. Why useful for analytics:

- **Simple, intuitive model** for business users — joins follow obvious "fact ↔ dimension" patterns.
 - **Query performance:** Star joins are easy for optimizers to handle; columnar warehouses scan only the needed measures and use bitmap / zone-map indexes on dimensions.
 - **Aggregation-friendly:** Slice / dice / roll-up (e.g., total revenue by quarter by category) is straightforward.
 - **Stable schema:** Dimensions evolve slowly; facts grow append-only — fits ETL/ELT pipelines and BI tools (cubes, Tableau, Power BI) cleanly.
-

End of study packet.